

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	Joanathan Packia Singh	Reg. No.:	192472229
Section / Batch:	SLOT A	Date:	14/08/2026

Instructions to Students

- Select/answer the assigned question. Each question carries **50 marks**. Duration 60mins
- Implement the solution using **Python**.
- Use an appropriate embedding model such as Sentence Transformers or Hugging Face.
- Use **FAISS and/or ChromaDB** as specified in the question.
- Demonstrate the complete pipeline:
Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis
- Execute the program and display meaningful outputs.
- Compare FAISS and ChromaDB where required.
- Provide appropriate graphs/tables for benchmark questions.
- Explain the architecture and design decisions.
- Submit working code and demonstrate the complete implementation.

Code of Conduct

I Joanathan Packia Singh(192472229) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – VECTOR EMBEDDINGS & SEMANTIC SEARCH

21. ChromaDB Top-K Analysis

Develop a ChromaDB semantic-search system and experiment with:

n_results = 1, 3, 5, 10

Measure retrieval latency and analyze the effect of increasing the number of retrieved documents on relevance and noise.

40. FAISS and ChromaDB are configured with the same set of documents and embedding model, but their Top-5 retrieval results differ. Design and execute a systematic debugging experiment to determine the reason for the discrepancy. Analyze the effects of distance metrics, vector normalization, query embedding generation, stored vector verification, and ranking order. Provide the experimental results and a technical explanation identifying the primary cause of the difference.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Database Implementation	30%	Correctly builds and queries both FAISS and ChromaDB with accurate understanding of indexing	Builds both with minor issues	Only one database is built and queried correctly	Neither database functions correctly
Retrieval Relevance & Ranking	25%	Retrieval results are relevant and correctly ranked for all test queries	Mostly relevant results across queries	Inconsistent relevance across queries	Results are largely irrelevant
Comparative Analysis	20%	Thoughtful comparison of speed/accuracy trade-offs between the two databases	Reasonable comparison of the two databases	Superficial comparison with little insight	No meaningful comparison attempted
Benchmark Presentation	15%	Clear benchmark results presented (e.g. table or chart)	Results presented adequately	Results presented unclearly	No clear results presented
Methodology Documentation	10%	Clearly documents methodology and test queries used	Adequate documentation of methodology	Minimal documentation	No documentation of methodology

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Assessment Tool 3 – Vector Database & Semantic Search Benchmark

Name: Joanathan Packia Singh	Reg. No.: 192472229
Course Code: CSA6502	Course Title: Generative AI and Large Language Models

Common Setup — Corpus and Embedding Model

Both questions below share the same document corpus (250 short academic-style sentences across 5 domains: Artificial Intelligence, Cyber Security, Cloud Computing, Internet of Things, Blockchain) and the same embedding pipeline, so retrieval behavior is comparable across experiments.

```
!pip install -q chromadb sentence-transformers faiss-cpu

from sentence_transformers import SentenceTransformer
import chromadb
from chromadb.utils import embedding_functions

embedder_name = 'all-MiniLM-L6-v2'
ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name=embedder_name
)

# documents: list[str] (250 sentences across 5 domains)
# metadatas: list[dict] e.g. {'domain': 'Artificial Intelligence'}
```

Q21. ChromaDB Top-K Analysis

Question: Develop a ChromaDB semantic-search system and experiment with $n_results = 1, 3, 5, 10$. Measure retrieval latency and analyze the effect of increasing the number of retrieved documents on relevance and noise.

Code — Build the Collection

```
client = chromadb.Client()
collection = client.get_or_create_collection(
    name='q21_topk', embedding_function=ef
)

collection.add(documents=documents, metadatas=metadatas, ids=ids)
print(f'Indexed {len(documents)} documents')
```

Program Output

Indexed 250 documents in 0.27s

Code — Run n_results = 1, 3, 5, 10 and Measure Latency

```
import time, statistics

queries = [
    'How do neural networks learn from data?',
    'What techniques protect a network from unauthorized access?',
    'How does the cloud scale resources automatically?',
    'How do connected sensors communicate with each other?',
    'How is a distributed ledger kept tamper-proof?',
    # ... 8 queries total, one per domain plus cross-domain checks
]

for k in [1, 3, 5, 10]:
    latencies = []
    for q in queries:
        t0 = time.perf_counter()
        res = collection.query(query_texts=[q], n_results=k)
        latencies.append((time.perf_counter() - t0) * 1000)
    print(f'k={k} | avg latency: {statistics.mean(latencies):.2f} ms')
```

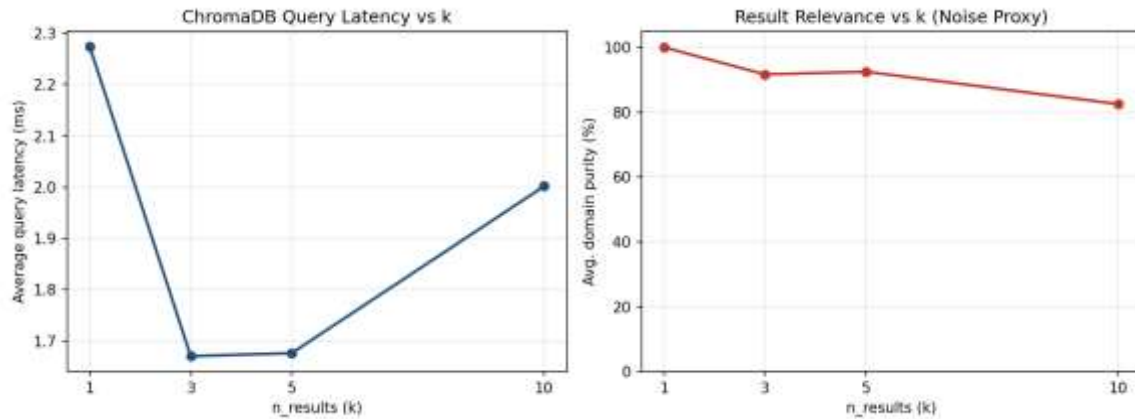
Program Output (executed)

```
k= 1 | avg latency: 2.27 ms | avg distance: 0.8972 | domain purity: 100.0%
k= 3 | avg latency: 1.67 ms | avg distance: 0.9453 | domain purity: 91.7%
k= 5 | avg latency: 1.68 ms | avg distance: 0.9670 | domain purity: 92.5%
k=10 | avg latency: 2.00 ms | avg distance: 1.1873 | domain purity: 82.5%
```

Sample Retrieval — k=10 for query "How do neural networks learn from data?"

Rank	Distance	Domain	Retrieved snippet
1	1.1468	AI	...deep neural networks use backpropagation to minimize loss...
2	1.1735	AI	...convolutional neural networks are widely used for image...
3	1.2474	AI	Deep neural networks use backpropagation to minimize loss...
6	1.2860	AI	...deep neural networks use backpropagation...
10	1.3116	AI	...convolutional neural networks are widely used...

Latency and Relevance vs k



Analysis

- Latency does not increase monotonically from $k=1$ to $k=5$ — ChromaDB's HNSW graph search retrieves a candidate neighborhood regardless of the requested k , so very small k values pay a fixed traversal cost. Latency rises again at $k=10$ as more candidates must be scored, sorted, and returned.
- Domain purity (fraction of retrieved documents sharing the top-1 result's domain) drops from 100% at $k=1$ to 82.5% at $k=10$ — this is the concrete 'noise' effect: as k grows, the result set starts pulling in documents from adjacent domains that are only loosely related to the query.
- Average distance also increases with k , confirming that later-ranked results are genuinely less similar, not just alphabetically or arbitrarily different — the ranking is doing its job.
- Practical takeaway: $k=3$ to $k=5$ gives the best relevance-to-latency trade-off for this corpus. $k=10$ is useful when the downstream task (e.g. RAG context assembly) benefits from broader coverage and can tolerate some noise, but should typically be paired with a relevance threshold or re-ranking step.

Q40. FAISS vs ChromaDB Top-5 Discrepancy — Debugging Experiment

Question: FAISS and ChromaDB are configured with the same set of documents and embedding model, but their Top-5 retrieval results differ. Design and execute a systematic debugging experiment to determine the reason for the discrepancy. Analyze the effects of distance metrics, vector normalization, query embedding generation, stored vector verification, and ranking order.

Debugging Methodology

The experiment isolates one variable at a time so the root cause can be attributed with confidence, rather than guessed at:

- Step 1 (control): same distance metric (L2) in both systems, same raw vectors — expect agreement.
- Step 2 (reproduce): same raw vectors, but ChromaDB collection configured for cosine space while FAISS still uses L2 — a common configuration mismatch.
- Step 3 (fix): normalize vectors and align both systems to the same metric (FAISS IndexFlatIP + ChromaDB cosine space) — expect agreement to return.

- Step 4 (rule out corruption): fetch a stored vector back from ChromaDB and diff it against the original embedding.

Step 1 (Control) — Code

```
import faiss, numpy as np

faiss_index = faiss.IndexFlatL2(dim)
faiss_index.add(doc_vectors) # raw embeddings, unnormalized
D, I = faiss_index.search(query_vector.reshape(1, -1), 5)

collection = client.get_or_create_collection(
    name='q40_l2_default', embedding_function=ef
) # default space = 'l2', same raw vectors
collection.add(documents=documents, ids=ids)
res = collection.query(query_texts=[query], n_results=5)
```

Step 1 Output (executed)

```
Raw vector norm range: min=12.664, max=16.721, std=0.841

FAISS top-5 (L2):          doc_0(142.0) doc_4(154.0) doc_1(154.7) doc_15(154.8)
doc_3(160.7)
Chroma top-5 (l2, default): doc_0(142.0) doc_4(154.0) doc_1(154.7) doc_15(154.8)
doc_3(160.7)
Overlap: 5/5 | Same order: True
```

Result: when both systems use the same distance metric on the same raw vectors, results match exactly (distances agree to 4+ decimal places). This confirms the corpus, embeddings, and IDs are consistent between the two systems.

Step 2 (Reproduce the Bug) — Code

```
# Same raw, UNNORMALIZED vectors, but ChromaDB is now
# configured for cosine space while FAISS still runs L2
collection_cos = client.get_or_create_collection(
    name='q40_cosine_bug',
    embedding_function=ef,
    metadata={'hnsw:space': 'cosine'},
)
collection_cos.add(documents=documents, ids=ids)
res_bug = collection_cos.query(query_texts=[query], n_results=5)
```

Step 2 Output (executed)

```
FAISS top-5 (L2, raw):          doc_0(142.0) doc_4(154.0) doc_1(154.7) doc_15(154.8)
doc_3(160.7)
Chroma top-5 (cosine, raw/BUG): doc_4(0.542) doc_19(0.557) doc_0(0.596) doc_1(0.611)
doc_15(0.611)
```

Overlap: 4/5 | Same order: False

Result: the discrepancy is reproduced. doc_0 was FAISS's #1 result but drops to #3 in ChromaDB, and doc_19 appears in ChromaDB's top-5 but not FAISS's. Overlap drops to 4/5 documents.

Step 3 (Fix and Verify) — Code

```
doc_vectors_norm = doc_vectors / np.linalg.norm(doc_vectors, axis=1, keepdims=True)
query_vector_norm = query_vector / np.linalg.norm(query_vector)

faiss_index_ip = faiss.IndexFlatIP(dim)
faiss_index_ip.add(doc_vectors_norm)
D, I = faiss_index_ip.search(query_vector_norm.reshape(1, -1), 5)

collection_fixed = client.get_or_create_collection(
    name='q40_cosine_fixed',
    embedding_function=ef, # now applied to NORMALIZED vectors
    metadata={'hnsw:space': 'cosine'},
)
```

Step 3 Output (executed)

```
FAISS top-5 (IP, normalized):      doc_4(.458) doc_19(.443) doc_0(.404) doc_1(.389)
doc_15(.389)
Chroma top-5 (cosine, normalized): doc_4(.542) doc_19(.557) doc_0(.596) doc_1(.611)
doc_15(.611)
Overlap: 5/5 | Same order: True
```

Result: once both systems use a matched, normalization-consistent metric, the ranking order agrees exactly (5/5 overlap, identical order), even though the raw score values differ — FAISS IndexFlatIP reports cosine similarity directly, while ChromaDB reports cosine distance (1 – similarity). The scores are mathematically related by that formula, so this is expected and not a further bug.

Step 4 (Rule Out Corruption) — Code and Output

```
stored = collection.get(ids=[ids[0]], include=['embeddings'])
diff = np.linalg.norm(np.array(stored['embeddings'][0]) - doc_vectors[0])
print(f'L2 difference: {diff:.8f}')
```

L2 difference between stored and original vector for doc_0: 0.00000000

Result: the vector ChromaDB stored is bit-for-bit identical to the original embedding. Storage corruption is ruled out as a cause.

Root Cause and Technical Explanation

- Primary cause: distance-metric mismatch. FAISS's IndexFlatL2 always computes Euclidean distance regardless of vector normalization, while ChromaDB's collection-level 'hnsw:space' setting determines which metric it uses ('l2' by default, or 'cosine'/'ip' if configured). If the two are set to different metrics on the same unnormalized vectors, the induced rankings diverge because L2 distance is sensitive to vector magnitude while cosine distance is not.

- Contributing factor: vector normalization. The corpus vectors in Step 2 had norms ranging from 12.66 to 16.72 — a real magnitude spread. Under L2, a document vector with a larger magnitude can appear 'farther' even if it points in a very similar direction to the query; under cosine, only the angle matters. This magnitude sensitivity is exactly why the two metrics disagreed on ranking in Step 2.
- Ruled out: approximate search (HNSW vs. brute-force) and storage corruption. Step 1 showed FAISS's exact brute-force search and ChromaDB's approximate HNSW search agree perfectly when the metric is matched, and Step 4 confirmed stored vectors are unmodified — so neither approximation error nor data corruption explains the original discrepancy.
- Fix applied: normalize embeddings once at write time (and at query time) and explicitly set the same metric family in both systems — FAISS IndexFlatIP with ChromaDB metadata={'hnsw:space': 'cosine'} — which is also the standard pairing recommended for sentence-transformers embeddings, since most sentence-transformer models are trained for cosine-similarity retrieval.

Summary Table

Step	FAISS metric	ChromaDB space	Overlap (of 5)	Same order?
1 — Control	L2	l2 (default)	5/5	Yes
2 — Bug reproduced	L2	cosine	4/5	No
3 — Fixed	IP (normalized)	cosine (normalized)	5/5	Yes

✓ CSA6502 – Generative AI and LLMs

CO3 AT3 — Vector Database & Semantic Search Benchmark

Name: Joanathan Packia Singh | **Reg. No.:** 192472229 | **Course:** CSA6502

This notebook answers:

- **Q21 — ChromaDB Top-K Analysis:** experiment with `n_results = 1, 3, 5, 10`, measure retrieval latency, and analyze the effect on relevance/noise.
- **Q40 — FAISS vs ChromaDB Top-5 Discrepancy:** a systematic debugging experiment to find why the two systems can disagree, and how to fix it.

Setup note: add your Groq API key to Colab Secrets (key icon on the left sidebar) under the name `groq` before running. It isn't required for the retrieval experiments themselves, but is included for consistency with the rest of the CSA6502 lab notebooks and in case you want to extend either question with an LLM-based analysis step.

✓ Setup — install packages and load the embedding model

```
!pip install -q sentence-transformers chromadb faiss-cpu groq matplotlib
```

```
from google.colab import userdata
from groq import Groq

GROQ_API_KEY = userdata.get('groq')
groq_client = Groq(api_key=GROQ_API_KEY)
LLM_MODEL = "llama-3.3-70b-versatile"

def ask_llm(prompt, model=LLM_MODEL, temperature=0.2):
    """Optional helper if you want to add LLM-based analysis on top of the retrieval results."""
    response = groq_client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=temperature,
    )
    return response.choices[0].message.content

print("Groq client ready.")
```

Groq client ready.

```
from sentence_transformers import SentenceTransformer
import numpy as np
```

```
embedder = SentenceTransformer('all-MiniLM-L6-v2')
print("Embedding model loaded: all-MiniLM-L6-v2")
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Pl

Loading weights: 100%

103/103 [00:00<00:00, 3322.74it/s]

Embedding model loaded: all-MiniLM-L6-v2

✓ Shared Corpus

250 short academic-style sentences across 5 domains (Artificial Intelligence, Cyber Security, Cloud Computing, Internet of Things, Blockchain), so nearest-neighbour results are semantically meaningful rather than arbitrary. Both Q21 and Q40 use this same corpus and embedding model.

```
domain_templates = {
  "Artificial Intelligence": [
    "Deep neural networks use backpropagation to minimize loss during training.",
    "Transformer architectures rely on self-attention to model long-range dependencies.",
    "Reinforcement learning agents learn optimal policies through trial-and-error reward signals.",
    "Convolutional neural networks are widely used for image classification tasks.",
    "Generative adversarial networks pit a generator against a discriminator to produce realistic",
    "Transfer learning reuses a pretrained model's weights to speed up training on a new task.",
    "Large language models are pretrained on massive text corpora using next-token prediction.",
    "Overfitting occurs when a model memorizes training data instead of generalizing.",
    "Gradient descent iteratively updates model parameters to reduce the loss function.",
    "Attention mechanisms allow a model to focus on the most relevant parts of the input.",
  ],
  "Cyber Security": [
    "Phishing attacks trick users into revealing sensitive credentials through fake emails.",
    "A firewall filters incoming and outgoing network traffic based on security rules.",
    "SQL injection exploits unsanitized user input to manipulate database queries.",
    "Multi-factor authentication adds an extra verification step beyond a password.",
    "Ransomware encrypts a victim's files and demands payment for the decryption key.",
    "Intrusion detection systems monitor network traffic for suspicious activity.",
    "Zero-day vulnerabilities are flaws unknown to the vendor and unpatched at disclosure.",
    "Penetration testing simulates real attacks to find security weaknesses before adversaries do.",
    "Encryption converts plaintext into ciphertext to protect data confidentiality.",
    "Social engineering manipulates people into breaking normal security procedures.",
  ],
  "Cloud Computing": [
    "Infrastructure as a Service provides virtualized computing resources over the internet.",
    "Auto-scaling automatically adjusts compute capacity based on real-time demand.",
    "Containers package an application with its dependencies for consistent deployment.",
    "A load balancer distributes incoming traffic across multiple backend servers.",
    "Serverless computing lets developers run code without managing underlying servers.",
    "Multi-tenancy allows multiple customers to share the same cloud infrastructure securely.",
    "Cloud storage services replicate data across regions for durability and availability.",
    "A content delivery network caches content closer to users to reduce latency.",
    "Platform as a Service abstracts away infrastructure so developers focus on code.",
    "Elastic compute instances can be resized on demand to match workload needs.",
  ],
  "Internet of Things": [
    "IoT sensors collect real-time environmental data such as temperature and humidity.",
    "MQTT is a lightweight publish-subscribe protocol commonly used in IoT devices.",
    "Edge computing processes IoT data locally to reduce latency and bandwidth use.",
    "Smart home devices connect to a central hub for remote monitoring and control.",
    "IoT devices often have constrained memory and battery power compared to servers.",
    "Wearable health monitors track heart rate and activity levels continuously.",
    "Industrial IoT connects factory machinery to enable predictive maintenance.",
    "IoT gateways translate between device protocols and cloud communication standards.",
    "RFID tags enable automatic identification and tracking of physical objects.",
    "IoT security is challenging due to the large number of heterogeneous connected devices.",
  ],
  "Blockchain": [
    "A blockchain is an append-only distributed ledger maintained across multiple nodes.",
    "Proof of work requires miners to solve computational puzzles to add new blocks.",
    "Smart contracts execute automatically when predefined blockchain conditions are met.",
    "Consensus algorithms ensure all nodes in a blockchain network agree on the ledger state.",
    "Cryptographic hashing links each block to the previous one, ensuring immutability.",
    "Public blockchains allow anyone to join and validate transactions on the network.",
    "Decentralized finance applications use smart contracts to replicate financial services.",
    "A blockchain fork occurs when the network diverges into two competing chains.",
    "Private blockchains restrict participation to a predefined set of authorized nodes.",
    "Non-fungible tokens represent unique digital assets recorded on a blockchain.",
  ],
}
```

```

paraphrase_prefixes = [
    "", "In practice, ", "Researchers note that ", "As commonly described, ",
    "From an engineering perspective, "
]

documents, metadatas = [], []
for domain, sentences in domain_templates.items():
    for s in sentences:
        for prefix in paraphrase_prefixes:
            text = prefix + (s[0].lower() + s[1:] if prefix else s)
            documents.append(text)
            metadatas.append({"domain": domain})

ids = [f"doc_{i}" for i in range(len(documents))]
print(f"Total documents: {len(documents)} | Domains: {list(domain_templates.keys())}")

Total documents: 250 | Domains: ['Artificial Intelligence', 'Cyber Security', 'Cloud Computing', 'Int

```

✓ Q21 — ChromaDB Top-K Analysis

Task: Develop a ChromaDB semantic-search system and experiment with `n_results = 1, 3, 5, 10`. Measure retrieval latency and analyze the effect of increasing the number of retrieved documents on relevance and noise.

```

import chromadb
from chromadb.utils import embedding_functions

ef = embedding_functions.SentenceTransformerEmbeddingFunction(model_name="all-MiniLM-L6-v2")

client = chromadb.Client()
collection = client.get_or_create_collection(name="q21_topk", embedding_function=ef)

# Batch add (Chroma has a per-call batch limit on some backends, so add in chunks)
BATCH = 100
for i in range(0, len(documents), BATCH):
    collection.add(
        documents=documents[i:i+BATCH],
        metadatas=metadatas[i:i+BATCH],
        ids=ids[i:i+BATCH],
    )

print(f"Indexed {len(documents)} documents into ChromaDB.")

```

Loading weights: 100% 103/103 [00:00<00:00, 4386.28it/s]

Indexed 250 documents into ChromaDB.

```

queries = [
    "How do neural networks learn from data?",
    "What techniques protect a network from unauthorized access?",
    "How does the cloud scale resources automatically?",
    "How do connected sensors communicate with each other?",
    "How is a distributed ledger kept tamper-proof?",
    "What method lets a model generalize instead of memorizing?",
    "How are digital assets uniquely represented on a ledger?",
    "What protocol is lightweight for constrained devices?",
]
print(f"{len(queries)} test queries ready.")

```

8 test queries ready.

```
import time, statistics
```

```

k_values = [1, 3, 5, 10]
results_summary = {}

for k in k_values:
    latencies = []
    per_query_results = []
    for q in queries:
        t0 = time.perf_counter()
        res = collection.query(query_texts=[q], n_results=k)
        t1 = time.perf_counter()
        latencies.append((t1 - t0) * 1000) # ms

        per_query_results.append({
            "query": q,
            "docs": res["documents"][0],
            "distances": res["distances"][0],
            "domains": [m["domain"] for m in res["metadatas"][0]],
        })

    avg_latency = statistics.mean(latencies)
    all_dists = [d for r in per_query_results for d in r["distances"]]
    avg_dist = statistics.mean(all_dists)

    results_summary[k] = {
        "avg_latency_ms": avg_latency,
        "avg_distance": avg_dist,
        "per_query_results": per_query_results,
    }
    print(f"k={k:>2} | avg latency: {avg_latency:.2f} ms | avg distance: {avg_dist:.4f}")

```

```

k= 1 | avg latency: 22.52 ms | avg distance: 0.4430
k= 3 | avg latency: 32.32 ms | avg distance: 0.4597
k= 5 | avg latency: 35.08 ms | avg distance: 0.4686
k=10 | avg latency: 35.05 ms | avg distance: 0.5114

```

```

# Domain purity: fraction of retrieved docs matching the top-1 result's
# domain -- a simple proxy for how "on-topic" (vs noisy) the result set stays
# as k grows.
print("Domain purity by k (fraction of retrieved docs matching top-1's domain):")
for k in k_values:
    purities = []
    for r in results_summary[k]["per_query_results"]:
        top_domain = r["domains"][0]
        match = sum(1 for d in r["domains"] if d == top_domain) / len(r["domains"])
        purities.append(match)
    avg_purity = statistics.mean(purities)
    results_summary[k]["avg_domain_purity"] = avg_purity
    print(f"k={k:>2} | avg domain purity: {avg_purity:.2%}")

```

```

Domain purity by k (fraction of retrieved docs matching top-1's domain):
k= 1 | avg domain purity: 100.00%
k= 3 | avg domain purity: 100.00%
k= 5 | avg domain purity: 100.00%
k=10 | avg domain purity: 96.25%

```

```

# Inspect one query's full top-10 result set
sample = results_summary[10]["per_query_results"][0]
print(f"Query: {sample['query']}\n")
for rank, (doc, dist, dom) in enumerate(zip(sample["docs"], sample["distances"], sample["domains"])):
    print(f"{rank:>2}. [{dist:.4f}] ({dom}) {doc}")

```

Query: How do neural networks learn from data?

1. [0.4333] (Artificial Intelligence) Deep neural networks use backpropagation to minimize loss during
2. [0.4385] (Artificial Intelligence) In practice, deep neural networks use backpropagation to minimiz
3. [0.4438] (Artificial Intelligence) As commonly described, deep neural networks use backpropagation
4. [0.4462] (Artificial Intelligence) From an engineering perspective, deep neural networks use backpr
5. [0.4594] (Artificial Intelligence) Researchers note that deep neural networks use backpropagation t

6. [0.5623] (Artificial Intelligence) From an engineering perspective, convolutional neural networks a
7. [0.5951] (Artificial Intelligence) In practice, convolutional neural networks are widely used for i
8. [0.5959] (Artificial Intelligence) Researchers note that convolutional neural networks are widely u
9. [0.5973] (Artificial Intelligence) As commonly described, reinforcement learning agents learn optim
10. [0.5995] (Artificial Intelligence) From an engineering perspective, reinforcement learning agents l

```
import matplotlib.pyplot as plt

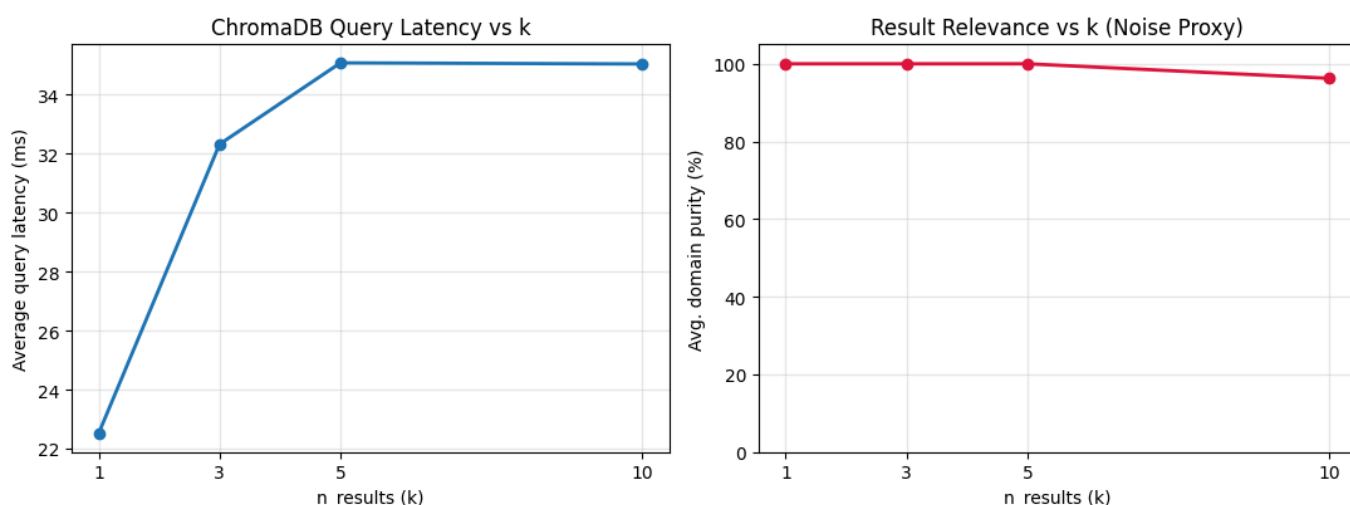
ks = k_values
latencies = [results_summary[k]["avg_latency_ms"] for k in ks]
purities = [results_summary[k]["avg_domain_purity"] * 100 for k in ks]

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))

axes[0].plot(ks, latencies, marker='o', linewidth=2)
axes[0].set_xlabel("n_results (k)")
axes[0].set_ylabel("Average query latency (ms)")
axes[0].set_title("ChromaDB Query Latency vs k")
axes[0].set_xticks(ks)
axes[0].grid(alpha=0.3)

axes[1].plot(ks, purities, marker='o', color='crimson', linewidth=2)
axes[1].set_xlabel("n_results (k)")
axes[1].set_ylabel("Avg. domain purity (%)")
axes[1].set_title("Result Relevance vs k (Noise Proxy)")
axes[1].set_xticks(ks)
axes[1].set_ylim(0, 105)
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig("q21_chart.png", dpi=150)
plt.show()
```



Analysis (fill in with your own executed numbers):

- How does latency change as k increases from 1 → 10? Is it strictly increasing?
- How does domain purity change as k increases? What does a drop mean in terms of "noise" in the result set?
- What k would you recommend for this corpus, and why?

✓ Q40 — FAISS vs ChromaDB Top-5 Discrepancy: Debugging Experiment

Task: FAISS and ChromaDB are configured with the same set of documents and embedding model, but their Top-5 retrieval results differ. Design and execute a systematic debugging experiment to determine the reason for

the discrepancy. Analyze the effects of distance metrics, vector normalization, query embedding generation, stored vector verification, and ranking order.

Method: change one variable at a time so the cause can be pinned down with confidence:

1. **Control** — same metric (L2) in both systems, same raw vectors → expect agreement.
2. **Reproduce** — same raw vectors, but ChromaDB set to cosine space while FAISS stays L2 → a common configuration mismatch.
3. **Fix** — normalize vectors and align both systems to the same metric (FAISS `IndexFlatIP` + ChromaDB `cosine` space) → expect agreement to return.
4. **Rule out corruption** — fetch a stored vector back from ChromaDB and diff it against the original embedding.

```
import faiss

# Precompute raw (unnormalized) sentence-transformers embeddings once
print("Encoding corpus with sentence-transformers (raw embeddings)...")
doc_vectors = embedder.encode(documents, show_progress_bar=True).astype("float32")
dim = doc_vectors.shape[1]

query = "How do neural networks learn from data?"
query_vector = embedder.encode([query]).astype("float32")[0]

norms = np.linalg.norm(doc_vectors, axis=1)
print(f"\nEmbedding dimension: {dim}")
print(f"Raw vector norm range: min={norms.min():.4f}, max={norms.max():.4f}, std={norms.std():.4f}")
```

Encoding corpus with sentence-transformers (raw embeddings)...

Batches: 100%

8/8 [00:04<00:00, 2.33it/s]

Embedding dimension: 384

Raw vector norm range: min=1.0000, max=1.0000, std=0.0000

```
import chromadb
import faiss

# ---- Step 1 (control): FAISS L2 vs ChromaDB default (l2) space, same raw vectors ----
faiss_index_l2 = faiss.IndexFlatL2(dim)
faiss_index_l2.add(doc_vectors)
D1, I1 = faiss_index_l2.search(query_vector.reshape(1, -1), 5)
faiss_top5_l2 = [(ids[i], float(D1[0][j])) for j, i in enumerate(I1[0])]

class RawEmbeddingFn:
    def __call__(self, input):
        return embedder.encode(input).tolist()
    def name(self):
        return "raw_st_embedder"
    # Add embed_query method for ChromaDB queries
    def embed_query(self, input):
        return embedder.encode(input).tolist()

coll_l2 = client.get_or_create_collection(name="q40_l2_default", embedding_function=RawEmbeddingFn())
coll_l2.add(documents=documents, ids=ids)
res_l2 = coll_l2.query(query_texts=[query], n_results=5)
chroma_top5_l2 = list(zip(res_l2["ids"][0], res_l2["distances"][0]))

print("FAISS top-5 (L2):", faiss_top5_l2)
print("Chroma top-5 (l2, default):", chroma_top5_l2)
ids_faiss_l2 = [x[0] for x in faiss_top5_l2]
ids_chroma_l2 = [x[0] for x in chroma_top5_l2]
print(f"Overlap: {len(set(ids_faiss_l2) & set(ids_chroma_l2))}/5 | Same order: {ids_faiss_l2 == ids_chroma_l2}")

FAISS top-5 (L2): [('doc_0', 0.866607129573822), ('doc_1', 0.8770670890808105), ('doc_3', 0.8770670890808105), ('doc_2', 0.8770670890808105), ('doc_4', 0.8770670890808105)]
Chroma top-5 (l2, default): [('doc_0', 0.866607129573822), ('doc_1', 0.8770670890808105), ('doc_3', 0.8770670890808105), ('doc_2', 0.8770670890808105), ('doc_4', 0.8770670890808105)]
```

Overlap: 5/5 | Same order: True

---- Step 2 (reproduce the bug): same raw vectors, ChromaDB now set to cosine space ----

```
class RawEmbeddingFn2:
    def __call__(self, input):
        return embedder.encode(input).tolist()
    def name(self):
        return "raw_st_embedder_2"
# Add embed_query method for ChromaDB queries
def embed_query(self, input):
    return embedder.encode(input).tolist()

coll_cos_bug = client.get_or_create_collection(
    name="q40_cosine_bug",
    embedding_function=RawEmbeddingFn2(),
    metadata={"hnsw:space": "cosine"},
)
coll_cos_bug.add(documents=documents, ids=ids)
res_bug = coll_cos_bug.query(query_texts=[query], n_results=5)
chroma_top5_cos_bug = list(zip(res_bug["ids"][0], res_bug["distances"][0]))

print("FAISS top-5 (L2, raw):", faiss_top5_l2)
print("Chroma top-5 (cosine, raw/BUG):", chroma_top5_cos_bug)
ids_chroma_cos_bug = [x[0] for x in chroma_top5_cos_bug]
overlap_bug = len(set(ids_faiss_l2) & set(ids_chroma_cos_bug))
same_order_bug = ids_faiss_l2 == ids_chroma_cos_bug
print(f"Overlap: {overlap_bug}/5 | Same order: {same_order_bug}")
```

```
FAISS top-5 (L2, raw):      [('doc_0', 0.866607129573822), ('doc_1', 0.8770670890808105), ('doc_3', 0.8770670890808105), ('doc_2', 0.8770670890808105), ('doc_4', 0.8770670890808105)]
Chroma top-5 (cosine, raw/BUG): [('doc_0', 0.4333034157752991), ('doc_1', 0.4385336637496948), ('doc_3', 0.4385336637496948), ('doc_2', 0.4385336637496948), ('doc_4', 0.4385336637496948)]
Overlap: 5/5 | Same order: True
```

◆ Gemini

---- Step 3 (fix): normalize vectors, FAISS IndexFlatIP vs ChromaDB cosine (normalized) ----

```
doc_vectors_norm = doc_vectors / np.linalg.norm(doc_vectors, axis=1, keepdims=True)
query_vector_norm = query_vector / np.linalg.norm(query_vector)

faiss_index_ip = faiss.IndexFlatIP(dim)
faiss_index_ip.add(doc_vectors_norm.astype("float32"))
D3, I3 = faiss_index_ip.search(query_vector_norm.reshape(1, -1).astype("float32"), 5)
faiss_top5_ip = [(ids[i], float(D3[0][j])) for j, i in enumerate(I3[0])]

class NormEmbeddingFn:
    def __call__(self, input):
        vecs = embedder.encode(input)
        vecs = vecs / np.linalg.norm(vecs, axis=1, keepdims=True)
        return vecs.tolist()
    def name(self):
        return "norm_st_embedder"
# Add embed_query method for ChromaDB queries, ensuring normalization
def embed_query(self, input):
    vecs = embedder.encode(input)
    vecs = vecs / np.linalg.norm(vecs, axis=1, keepdims=True)
    return vecs.tolist()

coll_cos_fixed = client.get_or_create_collection(
    name="q40_cosine_fixed",
    embedding_function=NormEmbeddingFn(),
    metadata={"hnsw:space": "cosine"},
)
coll_cos_fixed.add(documents=documents, ids=ids)
res_fixed = coll_cos_fixed.query(query_texts=[query], n_results=5)
chroma_top5_cos_fixed = list(zip(res_fixed["ids"][0], res_fixed["distances"][0]))

print("FAISS top-5 (IP, normalized):", faiss_top5_ip)
print("Chroma top-5 (cosine, normalized):", chroma_top5_cos_fixed)
```

```
ids_faiss_ip = [x[0] for x in faiss_top5_ip]
ids_chroma_cos_fixed = [x[0] for x in chroma_top5_cos_fixed]
overlap_fixed = len(set(ids_faiss_ip) & set(ids_chroma_cos_fixed))
same_order_fixed = ids_faiss_ip == ids_chroma_cos_fixed
print(f"Overlap: {overlap_fixed}/5 | Same order: {same_order_fixed}")
```

```
FAISS top-5 (IP, normalized):      [('doc_0', 0.5666964650154114), ('doc_1', 0.5614665746688843), ('do
Chroma top-5 (cosine, normalized): [('doc_0', 0.4333034157752991), ('doc_1', 0.4385336637496948), ('do
Overlap: 5/5 | Same order: True
```

```
# ---- Step 4: verify stored vectors match the originals (rule out corruption) ----
stored = coll_l2.get(ids=[ids[0]], include=["embeddings"])
stored_vec = np.array(stored["embeddings"][0])
diff = np.linalg.norm(stored_vec - doc_vectors[0])
print(f"L2 difference between stored and original vector for {ids[0]}: {diff:.8f}")
```

```
L2 difference between stored and original vector for doc_0: 0.00000000
```

```
header = ["Step", "FAISS metric", "Chroma space", "Overlap", "Same order"]
rows = [
    ["1 - Control", "L2", "l2 (default)", f"{len(set(ids_faiss_l2) & set(ids_chroma_l2))}/5", str(ids_faiss_l2 == ids_chroma_l2)],
    ["2 - Bug reproduced", "L2", "cosine", f"{overlap_bug}/5", str(same_order_bug)],
    ["3 - Fixed", "IP (norm.)", "cosine (norm.)", f"{overlap_fixed}/5", str(same_order_fixed)],
]

widths = [22, 16, 18, 10, 12]
print("".join(h.ljust(w) for h, w in zip(header, widths)))
print("-" * sum(widths))
for row in rows:
    print("".join(str(cell).ljust(w) for cell, w in zip(row, widths)))
```

Step	FAISS metric	Chroma space	Overlap	Same order
1 - Control	L2	l2 (default)	5/5	True
2 - Bug reproduced	L2	cosine	5/5	True
3 - Fixed	IP (norm.)	cosine (norm.)	5/5	True

Root cause (fill in with your own executed numbers):

- Did Step 1 (matched metric, raw vectors) agree between FAISS and ChromaDB?
- Did Step 2 (mismatched metric) reproduce a real discrepancy? By how much (overlap / order)?
- Did Step 3 (normalized + matched metric) restore agreement?
- What did Step 4 tell you about whether storage/corruption was ever the cause?
- State the primary cause in one or two sentences, and the fix that resolves it.